ESCUELA POLITÉCNICA NACIONAL

FACULTAD DE INGENIERÍA EN SISTEMAS

MÉTODOS NUMÉRICOS (C2021)

TAREA 4 METODO DE LA BISECCIÓN

JUAN FRANCISCO PINTO SARMIENTO

GRUPO 1

FECHA DE ENTREGA 20 DE NOVIEMBRE DE 2021

En este caso hemos implementado en programación en lenguaje Python, utilizando el método de bisección en la resolución de ecuaciones numéricas.

Conjunto de ejercicios

1.- Use el método de bisección para encontrar soluciones precisas dentro del intervalo $[a, b]$ para $f(x) = 2x^3 - 3x^2 - 12x - 3$ si los valores iniciales

- $a = 0$ y $b = 1$
- $a = 1$ y $b = 2$
- $a = 2$ y $b = 3$

```
import numpy
import matplotlib.pyplot as
# Definimos la función
def f(x):
    return 2*x**3 - 3*x**2 - 12*x - 3
# Definimos el intervalo
a = 0
b = 1
print("Intervalo de búsqueda de la raíz: [{}, {}]".format(a, b))
# Definimos el error
error = 0.5*10**(-5)
print("Error: {}".format(error))
# Definimos el número de iteraciones
n = int(numpy.ceil(-numpy.log(error/(b-a))/numpy.log(2)))
print("Número de iteraciones: {}".format(n))
# Bucle para el método de bisección
for i in range(1, n+1):
    c = (a+b)/2
    if f(a)*f(c) < 0:
        b = c
    else:
        a = c
    print("Iteración {}: c = {}, f(c) = {}, a = {}, b = {}".format(i, c, f(c), a, b))
# Imprimimos la raíz
print("La raíz es: {}".format(c))
# Imprimimos el error
print("Error: {}".format(abs(f(c))))
Intervalo de búsqueda de la raíz: [0, 1]
Error: 0.5*10**(-5)
Número de iteraciones: 17
Iteración 1: c = 0.5, f(c) = -4.5, a = 0, b = 1
Iteración 2: c = 0.75, f(c) = -2.8125, a = 0.5, b = 1
Iteración 3: c = 0.875, f(c) = -0.9375, a = 0.75, b = 1
Iteración 4: c = 0.9375, f(c) = 0.3369140625, a = 0.875, b = 1
Iteración 5: c = 0.90625, f(c) = -0.3022216797, a = 0.875, b = 0.9375
Iteración 6: c = 0.921875, f(c) = 0.0166549683, a = 0.90625, b = 0.9375
Iteración 7: c = 0.9140625, f(c) = -0.1436309814, a = 0.90625, b = 0.921875
Iteración 8: c = 0.91796875, f(c) = -0.0633811951, a = 0.9140625, b = 0.921875
Iteración 9: c = 0.920703125, f(c) = -0.0233966112, a = 0.91796875, b = 0.921875
Iteración 10: c = 0.921484375, f(c) = -0.0033969164, a = 0.920703125, b = 0.921875
Iteración 11: c = 0.921875, f(c) = 0.0066967010, a = 0.921484375, b = 0.921875
Iteración 12: c = 0.921679688, f(c) = 0.0016499138, a = 0.921484375, b = 0.921875
Iteración 13: c = 0.921582031, f(c) = -0.0008734465, a = 0.921679688, b = 0.921875
Iteración 14: c = 0.921630859, f(c) = 0.0003882647, a = 0.921582031, b = 0.921875
Iteración 15: c = 0.921606445, f(c) = -0.0002425289, a = 0.921630859, b = 0.921875
Iteración 16: c = 0.921618652, f(c) = 0.0000728178, a = 0.921606445, b = 0.921875
Iteración 17: c = 0.921612549, f(c) = -0.0000848866, a = 0.921618652, b = 0.921875
La raíz es: 0.921612549
Error: 0.0000848866
```

2.- Use el método de bisección para encontrar soluciones precisas dentro del intervalo $[a, b]$ para la raíz de cada ecuación $f(x) = 0$ si

- $f(x) = \cos x - x$, $a = 0$, $b = 1$
- $f(x) = 2x\cos 2x - (x-2)^2 - 3$, $a = 2$, $b = 3$

```
import numpy
import matplotlib.pyplot as
# Definimos la función
def f(x):
    return 2*x*numpy.cos(2*x) - (x-2)**2 - 3
# Definimos el intervalo
a = 2
b = 3
print("Intervalo de búsqueda de la raíz: [{}, {}]".format(a, b))
# Definimos el error
error = 0.5*10**(-5)
print("Error: {}".format(error))
# Definimos el número de iteraciones
n = int(numpy.ceil(-numpy.log(error/(b-a))/numpy.log(2)))
print("Número de iteraciones: {}".format(n))
# Bucle para el método de bisección
for i in range(1, n+1):
    c = (a+b)/2
    if f(a)*f(c) < 0:
        b = c
    else:
        a = c
    print("Iteración {}: c = {}, f(c) = {}, a = {}, b = {}".format(i, c, f(c), a, b))
# Imprimimos la raíz
print("La raíz es: {}".format(c))
# Imprimimos el error
print("Error: {}".format(abs(f(c))))
Intervalo de búsqueda de la raíz: [2, 3]
Error: 0.5*10**(-5)
Número de iteraciones: 17
Iteración 1: c = 2.5, f(c) = -0.5, a = 2, b = 3
Iteración 2: c = 2.75, f(c) = 0.8125, a = 2.5, b = 3
Iteración 3: c = 2.625, f(c) = 0.1328125, a = 2.5, b = 2.75
Iteración 4: c = 2.5625, f(c) = -0.1826171875, a = 2.625, b = 2.75
Iteración 5: c = 2.59375, f(c) = -0.0254516602, a = 2.5625, b = 2.75
Iteración 6: c = 2.609375, f(c) = 0.0540771484, a = 2.59375, b = 2.75
Iteración 7: c = 2.601562, f(c) = 0.0142211914, a = 2.59375, b = 2.609375
Iteración 8: c = 2.597656, f(c) = -0.0056152344, a = 2.601562, b = 2.609375
Iteración 9: c = 2.599609, f(c) = 0.0043029785, a = 2.597656, b = 2.609375
Iteración 10: c = 2.598633, f(c) = -0.0006866455, a = 2.599609, b = 2.609375
Iteración 11: c = 2.599121, f(c) = 0.0017967224, a = 2.598633, b = 2.609375
Iteración 12: c = 2.598877, f(c) = 0.0005550194, a = 2.598633, b = 2.599121
Iteración 13: c = 2.598755, f(c) = -0.0000657845, a = 2.598877, b = 2.599121
Iteración 14: c = 2.598816, f(c) = 0.0002446175, a = 2.598755, b = 2.599121
Iteración 15: c = 2.598785, f(c) = 0.0000894165, a = 2.598816, b = 2.599121
Iteración 16: c = 2.598770, f(c) = 0.0000118351, a = 2.598785, b = 2.599121
Iteración 17: c = 2.598762, f(c) = -0.0000273895, a = 2.598770, b = 2.599121
La raíz es: 2.598762
Error: 0.0000273895
```

3.- Use el método de bisección para encontrar soluciones precisas dentro de 10^-5 para la raíz de cada ecuación $f(x) = 0$ si

- $f(x) = 3x + 5$
- $f(x) = x^2 - 6x + 9$
- $f(x) = x^3 - x^2 - 1$
- $f(x) = e^{6x} + 3x^2 - 7x - 30$

```
import numpy
import matplotlib.pyplot as
# Definimos la función
def f(x):
    return 3*x + 5
# Definimos el intervalo
a = -2
b = 0
print("Intervalo de búsqueda de la raíz: [{}, {}]".format(a, b))
# Definimos el error
error = 0.5*10**(-5)
print("Error: {}".format(error))
# Definimos el número de iteraciones
n = int(numpy.ceil(-numpy.log(error/(b-a))/numpy.log(2)))
print("Número de iteraciones: {}".format(n))
# Bucle para el método de bisección
for i in range(1, n+1):
    c = (a+b)/2
    if f(a)*f(c) < 0:
        b = c
    else:
        a = c
    print("Iteración {}: c = {}, f(c) = {}, a = {}, b = {}".format(i, c, f(c), a, b))
# Imprimimos la raíz
print("La raíz es: {}".format(c))
# Imprimimos el error
print("Error: {}".format(abs(f(c))))
Intervalo de búsqueda de la raíz: [-2, 0]
Error: 0.5*10**(-5)
Número de iteraciones: 17
Iteración 1: c = -1.0, f(c) = 2.0, a = -2, b = 0
Iteración 2: c = -1.5, f(c) = -0.5, a = -1.5, b = 0
Iteración 3: c = -1.25, f(c) = 0.75, a = -1.5, b = -1.0
Iteración 4: c = -1.375, f(c) = 0.125, a = -1.5, b = -1.25
Iteración 5: c = -1.4375, f(c) = -0.1875, a = -1.375, b = -1.25
Iteración 6: c = -1.40625, f(c) = -0.0313, a = -1.4375, b = -1.25
Iteración 7: c = -1.390625, f(c) = 0.0469, a = -1.40625, b = -1.25
Iteración 8: c = -1.398438, f(c) = 0.0078, a = -1.390625, b = -1.25
Iteración 9: c = -1.402344, f(c) = -0.0117, a = -1.398438, b = -1.25
Iteración 10: c = -1.400391, f(c) = -0.0020, a = -1.402344, b = -1.25
Iteración 11: c = -1.399414, f(c) = 0.0029, a = -1.400391, b = -1.25
Iteración 12: c = -1.399902, f(c) = 0.0005, a = -1.399414, b = -1.25
Iteración 13: c = -1.400146, f(c) = -0.0007, a = -1.399902, b = -1.25
Iteración 14: c = -1.400024, f(c) = -0.0001, a = -1.400146, b = -1.25
Iteración 15: c = -1.399963, f(c) = 0.0002, a = -1.399902, b = -1.400146
Iteración 16: c = -1.399994, f(c) = 0.0000, a = -1.399963, b = -1.400146
Iteración 17: c = -1.400009, f(c) = -0.0000, a = -1.399994, b = -1.400146
La raíz es: -1.400009
Error: 0.0000
import numpy
import matplotlib.pyplot as
# Definimos la función
def f(x):
    return x**2 - 6*x + 9
# Definimos el intervalo
a = 2
b = 4
print("Intervalo de búsqueda de la raíz: [{}, {}]".format(a, b))
# Definimos el error
error = 0.5*10**(-5)
print("Error: {}".format(error))
# Definimos el número de iteraciones
n = int(numpy.ceil(-numpy.log(error/(b-a))/numpy.log(2)))
print("Número de iteraciones: {}".format(n))
# Bucle para el método de bisección
for i in range(1, n+1):
    c = (a+b)/2
    if f(a)*f(c) < 0:
        b = c
    else:
        a = c
    print("Iteración {}: c = {}, f(c) = {}, a = {}, b = {}".format(i, c, f(c), a, b))
# Imprimimos la raíz
print("La raíz es: {}".format(c))
# Imprimimos el error
print("Error: {}".format(abs(f(c))))
Intervalo de búsqueda de la raíz: [2, 4]
Error: 0.5*10**(-5)
Número de iteraciones: 17
Iteración 1: c = 3.0, f(c) = 0.0, a = 2, b = 4
Iteración 2: c = 2.5, f(c) = 0.25, a = 2, b = 3.0
Iteración 3: c = 2.75, f(c) = 0.0625, a = 2.5, b = 3.0
Iteración 4: c = 2.875, f(c) = 0.0156, a = 2.75, b = 3.0
Iteración 5: c = 2.9375, f(c) = 0.0039, a = 2.875, b = 3.0
Iteración 6: c = 2.9688, f(c) = 0.0010, a = 2.9375, b = 3.0
Iteración 7: c = 2.9844, f(c) = 0.0002, a = 2.9688, b = 3.0
Iteración 8: c = 2.9922, f(c) = 0.0000, a = 2.9844, b = 3.0
Iteración 9: c = 2.9961, f(c) = 0.0000, a = 2.9922, b = 3.0
Iteración 10: c = 2.9980, f(c) = 0.0000, a = 2.9961, b = 3.0
Iteración 11: c = 2.9990, f(c) = 0.0000, a = 2.9980, b = 3.0
Iteración 12: c = 2.9995, f(c) = 0.0000, a = 2.9990, b = 3.0
Iteración 13: c = 2.9998, f(c) = 0.0000, a = 2.9995, b = 3.0
Iteración 14: c = 2.9999, f(c) = 0.0000, a = 2.9998, b = 3.0
Iteración 15: c = 3.0000, f(c) = 0.0000, a = 2.9999, b = 3.0
Iteración 16: c = 3.0000, f(c) = 0.0000, a = 3.0000, b = 3.0
Iteración 17: c = 3.0000, f(c) = 0.0000, a = 3.0000, b = 3.0
La raíz es: 3.0000
Error: 0.0000
import numpy
import matplotlib.pyplot as
# Definimos la función
def f(x):
    return x**3 - x**2 - 1
# Definimos el intervalo
a = 1
b = 2
print("Intervalo de búsqueda de la raíz: [{}, {}]".format(a, b))
# Definimos el error
error = 0.5*10**(-5)
print("Error: {}".format(error))
# Definimos el número de iteraciones
n = int(numpy.ceil(-numpy.log(error/(b-a))/numpy.log(2)))
print("Número de iteraciones: {}".format(n))
# Bucle para el método de bisección
for i in range(1, n+1):
    c = (a+b)/2
    if f(a)*f(c) < 0:
        b = c
    else:
        a = c
    print("Iteración {}: c = {}, f(c) = {}, a = {}, b = {}".format(i, c, f(c), a, b))
# Imprimimos la raíz
print("La raíz es: {}".format(c))
# Imprimimos el error
print("Error: {}".format(abs(f(c))))
Intervalo de búsqueda de la raíz: [1, 2]
Error: 0.5*10**(-5)
Número de iteraciones: 17
Iteración 1: c = 1.5, f(c) = 0.875, a = 1, b = 1.5
Iteración 2: c = 1.25, f(c) = 0.2969, a = 1, b = 1.25
Iteración 3: c = 1.125, f(c) = -0.0515, a = 1.125, b = 1.25
Iteración 4: c = 1.188, f(c) = 0.1162, a = 1.125, b = 1.188
Iteración 5: c = 1.156, f(c) = 0.0312, a = 1.125, b = 1.156
Iteración 6: c = 1.141, f(c) = -0.0105, a = 1.141, b = 1.156
Iteración 7: c = 1.148, f(c) = 0.0102, a = 1.141, b = 1.148
Iteración 8: c = 1.145, f(c) = -0.0002, a = 1.145, b = 1.148
Iteración 9: c = 1.146, f(c) = 0.0049, a = 1.145, b = 1.146
Iteración 10: c = 1.146, f(c) = 0.0023, a = 1.145, b = 1.146
Iteración 11: c = 1.146, f(c) = 0.0010, a = 1.145, b = 1.146
Iteración 12: c = 1.146, f(c) = 0.0004, a = 1.145, b = 1.146
Iteración 13: c = 1.146, f(c) = 0.0001, a = 1.145, b = 1.146
Iteración 14: c = 1.146, f(c) = 0.0000, a = 1.145, b = 1.146
Iteración 15: c = 1.146, f(c) = 0.0000, a = 1.145, b = 1.146
Iteración 16: c = 1.146, f(c) = 0.0000, a = 1.145, b = 1.146
Iteración 17: c = 1.146, f(c) = 0.0000, a = 1.145, b = 1.146
La raíz es: 1.146
Error: 0.0000
import numpy
import matplotlib.pyplot as
# Definimos la función
def f(x):
    return numpy.exp(6*x) + 3*x**2 - 7*x - 30
# Definimos el intervalo
a = 1
b = 2
print("Intervalo de búsqueda de la raíz: [{}, {}]".format(a, b))
# Definimos el error
error = 0.5*10**(-5)
print("Error: {}".format(error))
# Definimos el número de iteraciones
n = int(numpy.ceil(-numpy.log(error/(b-a))/numpy.log(2)))
print("Número de iteraciones: {}".format(n))
# Bucle para el método de bisección
for i in range(1, n+1):
    c = (a+b)/2
    if f(a)*f(c) < 0:
        b = c
    else:
        a = c
    print("Iteración {}: c = {}, f(c) = {}, a = {}, b = {}".format(i, c, f(c), a, b))
# Imprimimos la raíz
print("La raíz es: {}".format(c))
# Imprimimos el error
print("Error: {}".format(abs(f(c))))
Intervalo de búsqueda de la raíz: [1, 2]
Error: 0.5*10**(-5)
Número de iteraciones: 17
Iteración 1: c = 1.5, f(c) = 1.4817, a = 1, b = 1.5
Iteración 2: c = 1.25, f(c) = -0.7183, a = 1.25, b = 1.5
Iteración 3: c = 1.375, f(c) = 0.3817, a = 1.25, b = 1.375
Iteración 4: c = 1.313, f(c) = -0.1683, a = 1.313, b = 1.375
Iteración 5: c = 1.344, f(c) = 0.0817, a = 1.313, b = 1.344
Iteración 6: c = 1.328, f(c) = -0.0433, a = 1.328, b = 1.344
Iteración 7: c = 1.336, f(c) = 0.0192, a = 1.328, b = 1.336
Iteración 8: c = 1.332, f(c) = -0.0121, a = 1.332, b = 1.336
Iteración 9: c = 1.334, f(c) = 0.0035, a = 1.332, b = 1.334
Iteración 10: c = 1.333, f(c) = -0.0043, a = 1.333, b = 1.334
Iteración 11: c = 1.333, f(c) = -0.0004, a = 1.333, b = 1.334
Iteración 12: c = 1.333, f(c) = 0.0015, a = 1.333, b = 1.333
Iteración 13: c = 1.333, f(c) = 0.0005, a = 1.333, b = 1.333
Iteración 14: c = 1.333, f(c) = 0.0000, a = 1.333, b = 1.333
Iteración 15: c = 1.333, f(c) = -0.0002, a = 1.333, b = 1.333
Iteración 16: c = 1.333, f(c) = -0.0000, a = 1.333, b = 1.333
Iteración 17: c = 1.333, f(c) = 0.0000, a = 1.333, b = 1.333
La raíz es: 1.333
Error: 0.0000
```

4.- Use el método de bisección para encontrar soluciones precisas dentro de 10^-4 para la raíz de cada ecuación $f(x) = 0$ si

- $f(x) = \sqrt{x} - \cos x$
- $f(x) = x - (2^{-x})$

```
import numpy
import matplotlib.pyplot as
# Definimos la función
def f(x):
    return numpy.sqrt(x) - numpy.cos(x)
# Definimos el intervalo
a = 0
b = 1
print("Intervalo de búsqueda de la raíz: [{}, {}]".format(a, b))
# Definimos el error
error = 0.5*10**(-4)
print("Error: {}".format(error))
# Definimos el número de iteraciones
n = int(numpy.ceil(-numpy.log(error/(b-a))/numpy.log(2)))
print("Número de iteraciones: {}".format(n))
# Bucle para el método de bisección
for i in range(1, n+1):
    c = (a+b)/2
    if f(a)*f(c) < 0:
        b = c
    else:
        a = c
    print("Iteración {}: c = {}, f(c) = {}, a = {}, b = {}".format(i, c, f(c), a, b))
# Imprimimos la raíz
print("La raíz es: {}".format(c))
# Imprimimos el error
print("Error: {}".format(abs(f(c))))
Intervalo de búsqueda de la raíz: [0, 1]
Error: 0.5*10**(-4)
Número de iteraciones: 14
Iteración 1: c = 0.5, f(c) = -0.3776, a = 0.5, b = 1
Iteración 2: c = 0.75, f(c) = 0.1018, a = 0.5, b = 0.75
Iteración 3: c = 0.625, f(c) = -0.1308, a = 0.625, b = 0.75
Iteración 4: c = 0.688, f(c) = -0.0145, a = 0.688, b = 0.75
Iteración 5: c = 0.719, f(c) = 0.0437, a = 0.688, b = 0.719
Iteración 6: c = 0.703, f(c) = 0.0146, a = 0.688, b = 0.703
Iteración 7: c = 0.695, f(c) = -0.0000, a = 0.695, b = 0.703
Iteración 8: c = 0.699, f(c) = 0.0073, a = 0.695, b = 0.699
Iteración 9: c = 0.697, f(c) = 0.0036, a = 0.695, b = 0.697
Iteración 10: c = 0.696, f(c) = 0.0018, a = 0.695, b = 0.696
Iteración 11: c = 0.695, f(c) = 0.0009, a = 0.695, b = 0.695
Iteración 12: c = 0.695, f(c) = 0.0004, a = 0.695, b = 0.695
Iteración 13: c = 0.695, f(c) = 0.0002, a = 0.695, b = 0.695
Iteración 14: c = 0.695, f(c) = 0.0000, a = 0.695, b = 0.695
La raíz es: 0.695
Error: 0.0000
import numpy
import matplotlib.pyplot as
# Definimos la función
def f(x):
    return x - (2**(-x))
# Definimos el intervalo
a = 0
b = 1
print("Intervalo de búsqueda de la raíz: [{}, {}]".format(a, b))
# Definimos el error
error = 0.5*10**(-4)
print("Error: {}".format(error))
# Definimos el número de iteraciones
n = int(numpy.ceil(-numpy.log(error/(b-a))/numpy.log(2)))
print("Número de iteraciones: {}".format(n))
# Bucle para el método de bisección
for i in range(1, n+1):
    c = (a+b)/2
    if f(a)*f(c) < 0:
        b = c
    else:
        a = c
    print("Iteración {}: c = {}, f(c) = {}, a = {}, b = {}".format(i, c, f(c), a, b))
# Imprimimos la raíz
print("La raíz es: {}".format(c))
# Imprimimos el error
print("Error: {}".format(abs(f(c))))
Intervalo de búsqueda de la raíz: [0, 1]
Error: 0.5*10**(-4)
Número de iteraciones: 14
Iteración 1: c = 0.5, f(c) = 0.1464, a = 0, b = 0.5
Iteración 2: c = 0.25, f(c) = -0.0732, a = 0.25, b = 0.5
Iteración 3: c = 0.375, f(c) = 0.0335, a = 0.25, b = 0.375
Iteración 4: c = 0.313, f(c) = -0.0199, a = 0.313, b = 0.375
Iteración 5: c = 0.344, f(c) = 0.0067, a = 0.313, b = 0.344
Iteración 6: c = 0.328, f(c) = -0.0066, a = 0.328, b = 0.344
Iteración 7: c = 0.336, f(c) = 0.0000, a = 0.336, b = 0.344
Iteración 8: c = 0.340, f(c) = 0.0033, a = 0.336, b = 0.340
Iteración 9: c = 0.338, f(c) = 0.0017, a = 0.336, b = 0.338
Iteración 10: c = 0.337, f(c) = 0.0008, a = 0.336, b = 0.337
Iteración 11: c = 0.336, f(c) = 0.0004, a = 0.336, b = 0.336
Iteración 12: c = 0.336, f(c) = 0.0002, a = 0.336, b = 0.336
Iteración 13: c = 0.336, f(c) = 0.0000, a = 0.336, b = 0.336
Iteración 14: c = 0.336, f(c) = 0.0000, a = 0.336, b = 0.336
La raíz es: 0.336
Error: 0.0000
```

Ejercicios Aplicados

11.- La pendiente de la pista L tiene una pendiente constante de cero en la pendiente cero pasa. Considerar la figura siguiente. Cuando se mueve por la pendiente L, la pendiente de la parte inferior de la pista cambia. La pendiente de la parte superior de la pista cambia de cero a -0.5 a -1.0 a -1.5 y a -2.0 en la parte inferior de la pista. La pendiente de la parte superior de la pista está dada por $f(x) = -0.5x$ y la pendiente de la parte inferior de la pista está dada por $f(x) = -x + 2.5$. Si la pista tiene una longitud de 10 pies, ¿a qué distancia a lo largo de la pista está la pelota?

Tomando que $f = 0.5x + 2.5x^2 - 2.5x^3 + 0.5x^4 - 0.5x^5 + 0.5x^6 - 0.5x^7 + 0.5x^8 - 0.5x^9 + 0.5x^{10} - 10$ Tiene 10 pies de longitud, y que x es la distancia a lo largo de la pista, se tiene que:

$$f = 0.5x + 2.5x^2 - 2.5x^3 + 0.5x^4 - 0.5x^5 + 0.5x^6 - 0.5x^7 + 0.5x^8 - 0.5x^9 + 0.5x^{10} - 10$$

Planteamientos y datos del ejercicio

Intervalo: $[a, b] = [0.0, 10.0]$, $[0.0, 10.0]$

Error de corte: $\varepsilon = 10^{-5}$ (Tomando $\varepsilon_1 = 10^{-5}$)

Cantidad de cifras: $n \geq -\log(\varepsilon) - \log(b-a) / \log(2)$

$n \geq -\log(10^{-5}) - \log(10.0 - 0.0) / \log(2)$

$n \geq -\log(10^{-5}) - \log(10.0) / \log(2)$

Reemplazando: $n \geq -\log(10^{-5}) - \log(10) / \log(2)$

$n \geq 20 - 3.3219$

Resultado: Para encontrar $\varepsilon = 10^{-5}$, $n \geq 16.68$

Cantidad de iteraciones: $n \geq -\log(10^{-5}) / \log(2)$, $n \geq 16.68$

```
import numpy
import matplotlib.pyplot as
# Definimos la función
def f(x):
    return 0.5*x + 2.5*x**2 - 2.5*x**3 + 0.5*x**4 - 0.5*x**5 + 0.5*x**6 - 0.5*x**7 + 0.5*x**8 - 0.5*x**9 + 0.5*x**10 - 10
# Definimos el intervalo
a = 0.0
b = 10.0
print("Intervalo de búsqueda de la raíz: [{}, {}]".format(a, b))
# Definimos el error
error = 0.5*10**(-5)
print("Error: {}".format(error))
# Definimos el número de iteraciones
n = int(numpy.ceil(-numpy.log(error/(b-a))/numpy.log(2)))
print("Número de iteraciones: {}".format(n))
# Bucle para el método de bisección
for i in range(1, n+1):
    c = (a+b)/2
    if f(a)*f(c) < 0:
        b = c
    else:
        a = c
    print("Iteración {}: c = {}, f(c) = {}, a = {}, b = {}".format(i, c, f(c), a, b))
# Imprimimos la raíz
print("La raíz es: {}".format(c))
# Imprimimos el error
print("Error: {}".format(abs(f(c))))
Intervalo de búsqueda de la raíz: [0.0, 10.0]
Error: 0.5*10**(-5)
Número de iteraciones: 17
Iteración 1: c = 5.0, f(c) = 1.0E+06, a = 0.0, b = 10.0
Iteración 2: c = 2.5, f(c) = 1.0E+05, a = 0.0, b = 5.0
Iteración 3: c = 1.25, f(c) = 1.0E+04, a = 0.0, b = 2.5
Iteración 4: c = 0.625, f(c) = 1.0E+03, a = 0.0, b = 1.25
Iteración 5: c = 0.313, f(c) = 1.0E+02, a = 0.0, b = 0.625
Iteración 6: c = 0.156, f(c) = 1.0E+01, a = 0.0, b = 0.313
Iteración 7: c = 0.078, f(c) = 1.0E+00, a = 0.0, b = 0.156
Iteración 8: c = 0.039, f(c) = -1.0E-01, a = 0.039, b = 0.156
Iteración 9: c = 0.098, f(c) = 1.0E-01, a = 0.039, b = 0.098
Iteración 10: c = 0.068, f(c) = -1.0E-02, a = 0.068, b = 0.098
Iteración 11: c = 0.083, f(c) = 1.0E-02, a = 0.068, b = 0.083
Iteración 12: c = 0.076, f(c) = -1.0E-03, a = 0.076, b = 0.083
Iteración 13: c = 0.079, f(c) = 1.0E-03, a = 0.076, b = 0.079
Iteración 14: c = 0.078, f(c) = -1.0E-04, a = 0.078, b = 0.079
Iteración 15: c = 0.078, f(c) = 1.0E-04, a = 0.078, b = 0.078
Iteración 16: c = 0.078, f(c) = -1.0E-05, a = 0.078, b = 0.078
Iteración 17: c = 0.078, f(c) = 1.0E-05, a = 0.078, b = 0.078
La raíz es: 0.078
Error: 1.0E-05
import numpy
import matplotlib.pyplot as
# Definimos la función
def f(x):
    return 0.5*x + 2.5*x**2 - 2.5*x**3 + 0.5*x**4 - 0.5*x**5 + 0.5*x**6 - 0.5*x**7 + 0.5*x**8 - 0.5*x**9 + 0.5*x**10 - 10
# Definimos el intervalo
a = 0.0
b = 10.0
print("Intervalo de búsqueda de la raíz: [{}, {}]".format(a, b))
# Definimos el error
error = 0.5*10**(-5)
print("Error: {}".format(error))
# Definimos el número de iteraciones
n = int(numpy.ceil(-numpy.log(error/(b-a))/numpy.log(2)))
print("Número de iteraciones: {}".format(n))
# Bucle para el método de bisección
for i in range(1, n+1):
    c = (a+b)/2
    if f(a)*f(c) < 0:
        b = c
    else:
        a = c
    print("Iteración {}: c = {}, f(c) = {}, a = {}, b = {}".format(i, c, f(c), a, b))
# Imprimimos la raíz
print("La raíz es: {}".format(c))
# Imprimimos el error
print("Error: {}".format(abs(f(c))))
Intervalo de búsqueda de la raíz: [0.0, 10.0]
Error: 0.5*10**(-5)
Número de iteraciones: 17
Iteración 1: c = 5.0, f(c) = 1.0E+06, a = 0.0, b = 10.0
Iteración 2: c = 2.5, f(c) = 1.0E+05, a = 0.0, b = 5.0
Iteración 3: c = 1.25, f(c) = 1.0E+04, a = 0.0, b = 2.5
Iteración 4: c = 0.625, f(c) = 1.0E+03, a = 0.0, b = 1.25
Iteración 5: c = 0.313, f(c) = 1.0E+02, a = 0.0, b = 0.625
Iteración 6: c = 0.156, f(c) = 1.0E+01, a = 0.0, b = 0.313
Iteración 7: c = 0.078, f(c) = 1.0E+00, a = 0.0, b = 0.156
Iteración 8: c = 0.039, f(c) = -1.0E-01, a = 0.039, b = 0.156
Iteración 9: c = 0.098, f(c) = 1.0E-01, a = 0.039, b = 0.098
Iteración 10: c = 0.068, f(c) = -1.0E-02, a = 0.068, b = 0.098
Iteración 11: c = 0.083, f(c) = 1.0E-02, a = 0.068, b = 0.083
Iteración 12: c = 0.076, f(c) = -1.0E-03, a = 0.076, b = 0.083
Iteración 13: c = 0.079, f(c) = 1.0E-03, a = 0.076, b = 0.079
Iteración 14: c = 0.078, f(c) = -1.0E-04, a = 0.078, b = 0.079
Iteración 15: c = 0.078, f(c) = 1.0E-04, a = 0.078, b = 0.078
Iteración 16: c = 0.078, f(c) = -1.0E-05, a = 0.078, b = 0.078
Iteración 17: c = 0.078, f(c) = 1.0E-05, a = 0.078, b = 0.078
La raíz es: 0.078
Error: 1.0E-05
```

Ejercicios Teóricos

1.- Suponiendo que $f(x)$ representa una ecuación en una raíz p de multiplicidad m mayor que uno proporcionada con anterioridad, sea $h^{*}(x) = f(x)/f'(x)$ y $g^{*}(x) = 1 - f(x)f''(x)/f'(x)^2$ Demuestre que p es un cero simple de $h^{*}(x)$ y de $g^{*}(x)$.

$$h^{*}(x) = f(x)/f'(x) \quad g^{*}(x) = 1 - f(x)f''(x)/f'(x)^2$$

Planteamiento teórico: $f(x) = (x - p)^m$

Derivadas: $f'(x) = m(x - p)^{m-1}$

$f''(x) = m(m-1)(x - p)^{m-2}$

$h^{*}(x) = \frac{(x-p)^m}{m(x-p)^{m-1}}$

$g^{*}(x) = 1 - \frac{(x-p)^m \cdot m(m-1)(x-p)^{m-2}}{m(x-p)^{m-1}}$

$h^{*}(x) = \frac{(x-p)}{m}$

$g^{*}(x) = 1 - \frac{(x-p)(m-1)}{m}$

$g^{*}(x) = \frac{1}{m} + \frac{(m-1)}{m}$

Evaluando en la raíz:

$h^{*}(p) = 0$

$g^{*}(p) = \frac{1}{m} \neq 0$

Resultado final: p es cero simple de $h^{*}(x)$ y de $g^{*}(x)$

2.- Suponiendo que f es una función continua en $[a, b]$ y que $f(a)f(b) < 0$. Si p_n y q_n denotan dos aproximaciones a un cero de f en $[a, b]$ y $n \geq 1$, demuestre que $|p_n - q_n| \leq \frac{b-a}{2^n}$ cuando $n \geq 1$.

$$|p_n - p| \leq \frac{b-a}{2^n} \quad \text{cuando} \quad n \geq 1.$$

Planteamiento teórico: $|p - q| \leq \frac{b-a}{2^n}$

Desarrollo: $|p - q| \leq \frac{b-a}{2^n}$

$|p - q| \leq \frac{b-a}{2^n}$

$|p - q| \leq \frac{b-a}{2^n}$

$|p - q| \leq \frac{b-a}{2^n}$

$|p - q| \leq \frac{b-a}{2^n}$

Resultado final: $|p - q| \leq \frac{b-a}{2^n}$ cuando $n \geq 1$

```
import numpy
import matplotlib.pyplot as
# Definimos la función
def f(x):
    return 0.5*x + 2.5*x**2 - 2.5*x**3 + 0.5*x**4 - 0.5*x**5 + 0.5*x**6 - 0.5*x**7 + 0.5*x**8 - 0.5*x**9 + 0.5*x**10 - 10
# Definimos el intervalo
a = 0.0
b = 10.0
print("Intervalo de búsqueda de la raíz: [{}, {}]".format(a, b))
# Definimos el error
error = 0.5*10**(-5)
print("Error: {}".format(error))
# Definimos el número de iteraciones
n = int(numpy.ceil(-numpy.log(error/(b-a))/numpy.log(2)))
print("Número de iteraciones: {}".format(n))
# Bucle para el método de bisección
for i in range(1, n+1):
    c = (a+b)/2
    if f(a)*f(c) < 0:
        b = c
    else:
        a = c
    print("Iteración {}: c = {}, f(c) = {}, a = {}, b = {}".format(i, c, f(c), a, b))
# Imprimimos la raíz
print("La raíz es: {}".format(c))
# Imprimimos el error
print("Error: {}".format(abs(f(c))))
Intervalo de búsqueda de la raíz: [0.0, 10.0]
Error: 0.5*10**(-5)
Número de iteraciones: 17
Iteración 1: c = 5.0, f(c) = 1.0E+06, a = 0.0, b = 10.0
Iteración 2: c = 2.5, f(c) = 1.0E+05, a = 0.0, b = 5.0
Iteración 3: c = 1.25, f(c) = 1.0E+04, a = 0.0, b = 2.5
Iteración 4: c = 0.625, f(c) = 1.0E+03, a = 0.0, b = 1.25
Iteración 5: c = 0.313, f(c) = 1.0E+02, a = 0.0, b = 0.625
Iteración 6: c = 0.156, f(c) = 1.0E+01, a = 0.0, b = 0.313
Iteración 7: c = 0.078, f(c) = 1.0E+00, a = 0.0, b = 0.156
Iteración 8: c = 0.039, f(c) = -1.0E-01, a = 0.039, b = 0.156
Iteración 9: c = 0.098, f(c) = 1.0E-01, a = 0.039, b = 0.098
Iteración 10: c = 0.068, f(c) = -1.0E-02, a = 0.068, b = 0.098
Iteración 11: c = 0.083, f(c) = 1.0E-02, a = 0.068, b = 0.083
Iteración 12: c = 0.076, f(c) = -1.0E-03, a = 0.076, b = 0.083
Iteración 13: c = 0.079, f(c) = 1.0E-03, a = 0.076, b = 0.079
Iteración 14: c = 0.078, f(c) = -1.0E-04, a = 0.078, b = 0.079
Iteración 15: c = 0.078, f(c) = 1.0E-04, a = 0.078, b = 0.078
Iteración 16: c = 0.078, f(c) = -1.0E-05, a = 0.078, b = 0.078
Iteración 17: c = 0.078, f(c) = 1.0E-05, a = 0.078, b = 0.078
La raíz es: 0.078
Error: 1.0E-05
```

2

La función definida en $[0, \infty)$ dada por $f(x) = \sin x + \cos x + x$ tiene al menos un número de Hermite entre $x = 0$ y $x = 10$. Si f es dada de Hermite encuentre p.

- $a = 0$, $b = 10$
- $a = 0$, $b = 10$

```
import numpy
import matplotlib.pyplot as
# Definimos la función
def f(x):
    return numpy.sin(x) + numpy.cos(x) + x
# Definimos el intervalo
a = 0
b = 10
print("Intervalo de búsqueda de la raíz: [{}, {}]".format(a, b))
# Definimos el error
error = 0.5*10**(-5)
print("Error: {}".format(error))
# Definimos el número de iteraciones
n = int(numpy.ceil(-numpy.log(error/(b-a))/numpy.log(2)))
print("Número de iteraciones: {}".format(n))
# Buc
```